INVARIANT TESTING FOR SOROBAN

BREAK WATER

Hack Meridian / 2026 LISBON | GUAVA® Stellar

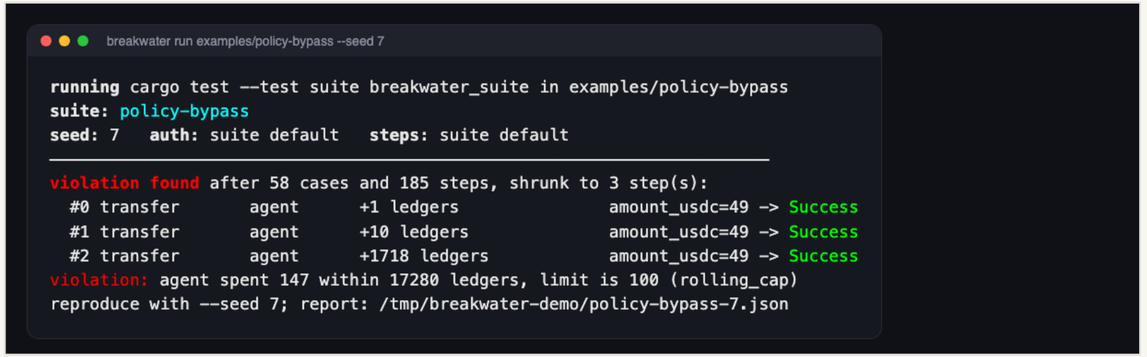
© 2026 HackMeridian · Lisbon

PITCH STEP 01 / 06

THE SOLUTION

Breakwater searches for the call sequence that breaks your Soroban invariants.

- Random multi-step sequences run against a local Soroban Env; every invariant is checked after every step.
- A failure is shrunk to its minimal steps and exported as a Rust regression test.
- Two attack packs for Stellar-specific risk: smart-account policies and TTL state archival.



👉 **Think:** Single-call tests prove each payment is fine. Only a sequence shows whether the account is.

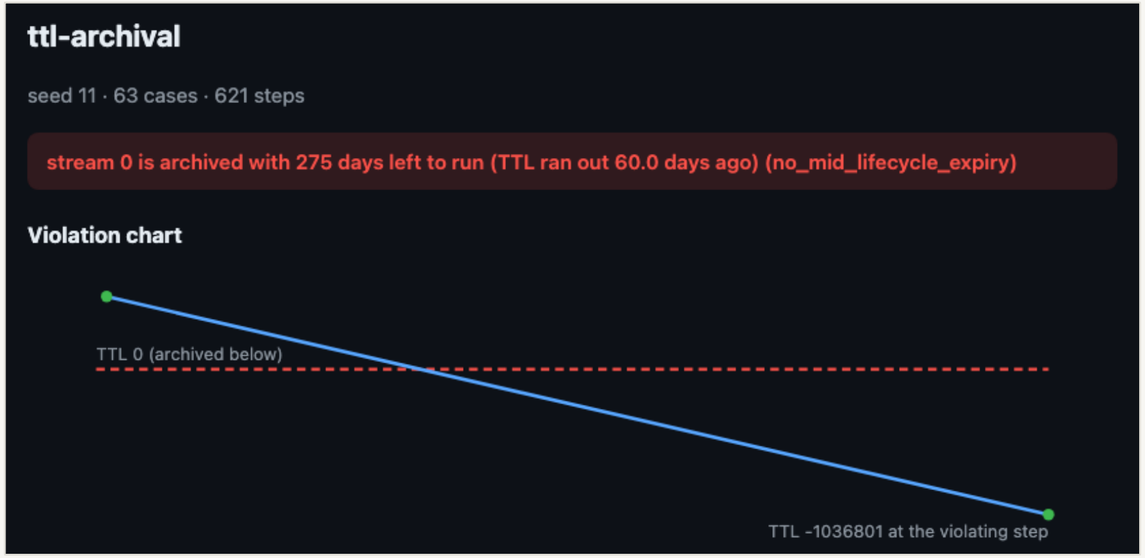
© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 02 / 06

PMF

Soroban has static analysis, AI review and formal verification, but no reusable stateful fuzzing.

- Agent wallets paying over x402 / MPP rely on smart-account policies; a policy that holds for one call may fail for five.
- Protocol 23 archived 478 contract entries with stale values; 84 were restored into live state before Protocol 24 fixed it.
- For every Soroban team shipping custom accounts or long-lived storage; a suite runs as cargo test.



👉 **Think:** The bugs that cost money here only appear across many transactions and long gaps.

© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 03 / 06

TECHNICAL WORKFLOW

Declare, generate, check, shrink and export: one cargo test.

- proptest-state-machine generates seeded sequences; each case restores a ledger snapshot instead of redeploying.
- Real auth mode signs every step with Ed25519, so OpenZeppelin's __check_auth and policies really run.
- TTL is read from ledger entries without touching them, so probing never auto-restores an archived entry.




```
//! Reproduction exported by `breakwater export` for suite `policy-bypass` with seed 7.
//! Replays the shrunk sequence through `common::replay_json` and asserts
//! that the same invariant fails at the same step.

mod common;

const STEPS: &str = r#"[
  {
    "handler": "transfer",
    "actor": "agent",
    "args": {
      "amount_usdc": 49
    },
    "gap": {
      "ledgers": 1,
      "seconds": 5
    }
  },
  {
    "handler": "transfer",
    "actor": "agent",
    "args": {
      "amount_usdc": 49
    },
    "gap": {
```

👉 **Think:** A finding under real authorization is a real bypass, not a mocking artifact.

© 2026 HackMeridian · Lisbon · #HackMeridian26

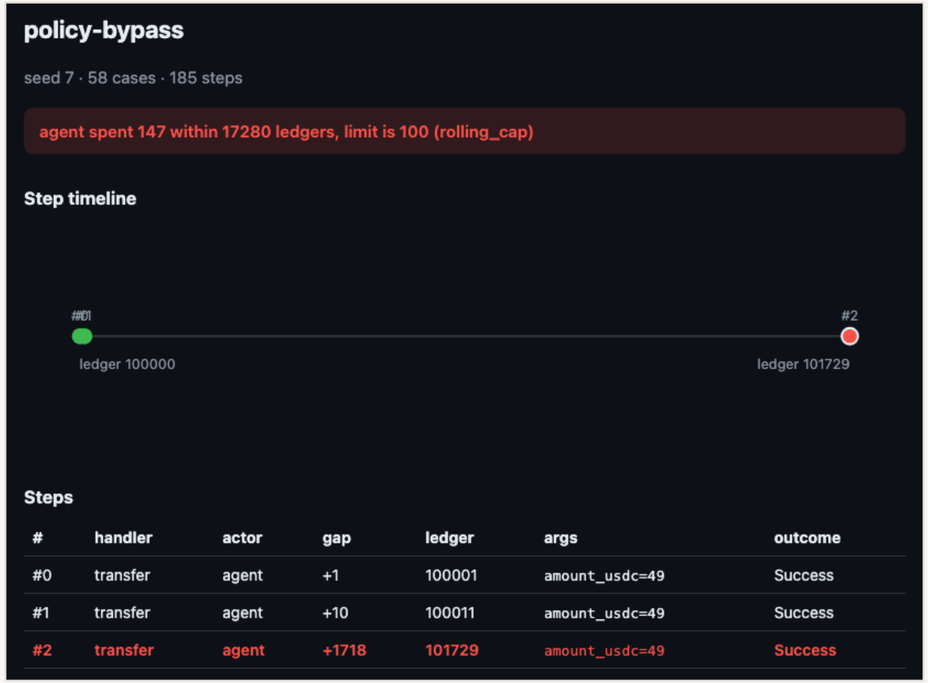
PITCH STEP 04 / 06

FINDINGS

The policy code is correct; the configuration is not. Every payment passes alone; the sequence does not.

- Daily window 1 728 ledgers, not 17 280: 49 → 49 → 49 USDC (+1 / +10 / +1718) is 147 in 24 h vs a 100 cap; up to 10× a day.
- No extend_ttl: “stream 0 is archived with 275 days left to run”.
- Unauthenticated bump_ttl keeps a stream alive 1.3 days past its end.
- After 90 quiet days one withdraw_many restores 5 entries against a budget of 3.

👉 **Think:** Breakwater found the three-step sequence that no single-call test can see.



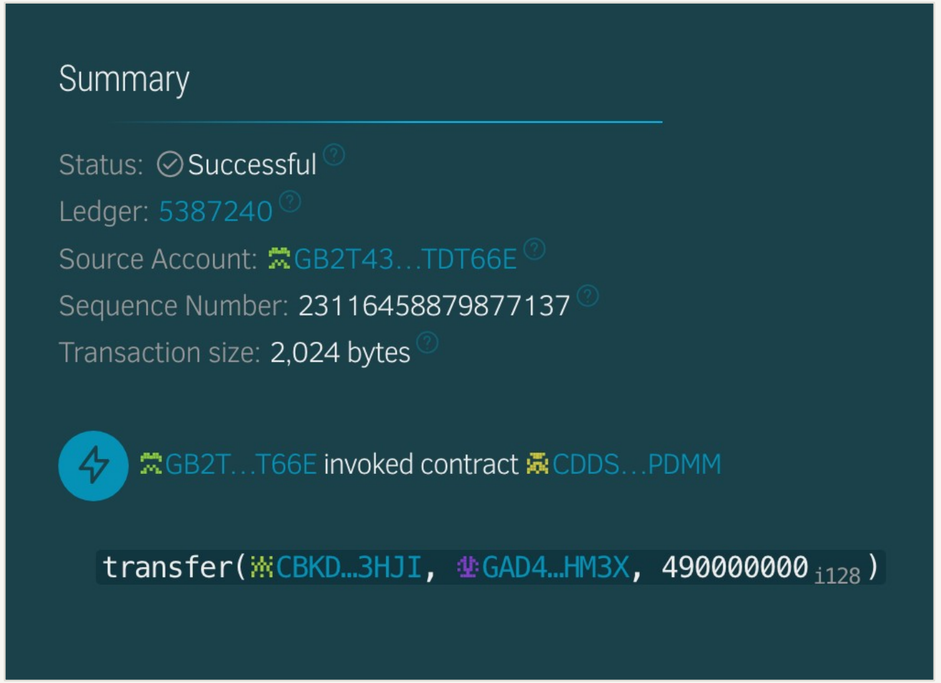
© 2026 HackMeridian · Lisbon · #HackMeridian26

PITCH STEP 05 / 06

TESTNET & PROGRESS

The shrunk bypass replayed on Stellar Testnet through the real OpenZeppelin account.

- Three agent-signed payments: 147 USDC in 3 537 ledgers (about 4.9 hours), all accepted by the policies.
- Seven contracts and five transactions on Testnet, each linked from the README.
- Built during the event: 11 commits on main, 10 merged PRs, 40 tests, CI on every PR.



👉 **Think:** Every number in this deck comes from a command anyone can rerun.

PITCH STEP 06 / 06

THE TEAM

Hannes Gao · Belvast Innovation · Karlsruhe, Germany

- Solo builder: framework design, Rust implementation, attack packs, Testnet replay.
- Repository: github.com/Belvast-Innovation/Breakwater (Apache-2.0).
- Next: rule-rotation probe, restoration-cost CI gate, XDR replay export; then crates.io and a GitHub Action.

👉 **Think:** Everything shown was built during the event; the commit history is the record.

`github.com/Belvast-Innovation/Breakwater`

© 2026 HackMeridian · Lisbon · #HackMeridian26